SÉANCE 1 · LIVRET ÉTUDIANT

Des couches qui ont du sens

Séparer présentation, métier et infrastructure, et appliquer SOLID à toute une application

Élément	Détail
Module	Architecture Hexagonale & DDD (A2627_4671) - Bachelor 3 Informatique - Développement
Séance	Séance 1 - lundi 19 octobre 2026, 9h-13h (4 h, face à face pédagogique)
Fil rouge	Le Fournil de Serris, boulangerie de quartier qui lance son Click & Collect (fictif)
Dossier de départ	`boulangerie-click-collect-seance1` (zip déposé sur la plateforme)
Prérequis	POO et SOLID à l'échelle d'une classe, TypeScript de base, Node.js 20+ installé.
TP de la séance	Pas de TP à rendre : l'étape 4 se fait en binôme et sert de point de départ à la séance 2.

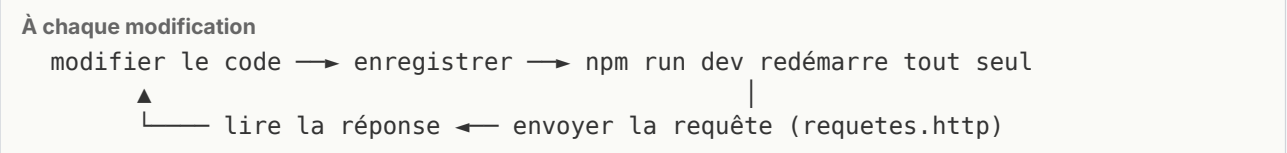
> Ce livret se suit pendant la séance. Si vous décrochez de l'oral, pas de panique : chaque étape est écrite pour que vous puissiez la refaire seul, à votre rythme. Les fiches-mémo vont droit au but : vous êtes en B3, le code n'est donné que là où il fait gagner du temps.

Comment utiliser ce livret

La séance compte **7 étapes**. Les étapes 1, 3, 5, 6 et 7 se font ensemble ; les étapes 2 et 4 se font **en binôme**, à votre clavier.

Rubrique	Ce qu'elle contient	Logique
À quoi ça sert	Le problème d'architecture qu'on veut résoudre, avant toute ligne de code.	EXPLIQUER
Ce que vous tapez	Les fichiers à créer ou modifier, et les commandes.	MODIFIER
Ce que vous devez voir	Le résultat observable : réponse de l'API, test, sortie du terminal.	TESTER
Le piège	L'erreur classique de l'étape, et comment la reconnaître.	-
À vous	Un petit exercice sans recopier, souvent avec un « pourquoi ».	MODIFIER + TESTER

La boucle à répéter tout le temps



> **Astuce**
> Les raccourcis sont notés Windows/Linux puis macOS : `Ctrl+Maj+P` / `Cmd+Maj+P`. Installez l'extension VS Code **REST Client** : un clic sur « Send Request » au-dessus de chaque requête du fichier `requetes.http`.

> **Et l'IA ?**
> Dans ce module, l'IA sert à **vous faire expliquer** (une erreur TypeScript, un principe), jamais à écrire à votre place. Deux réalisations sont **sans IA du tout** : le TP1 (séance 3) et le TP2 (séance 6).
>
> Les énoncés contiennent des marqueurs qui permettent de repérer un copier-coller intégral dans une IA. En soutenance, vous répondez de chaque ligne et de chaque diagramme que vous rendez.

Ce que contient ce livret

- Étapes 1 à 7 : du code « historique » à une architecture en couches, puis ses limites.
- Mémo express : les trois couches, SOLID à l'échelle d'une architecture, les commandes.

Avant de commencer : checklist

Vérification	Comment	OK ?
Node.js 20 ou plus	`node -v` affiche v20 ou plus	☐
Git installé	`git --version`	☐
Kit décompressé	Dossier `boulangerie-click-collect-seance1` dans Documents (pas dans Téléchargements), ouvert dans VS Code	☐

Vérification	Comment	OK ?
Extension REST Client	Onglet Extensions de VS Code, « REST Client » (Huachao Mao)	☐
IA désactivée par défaut	Copilot ou équivalent en pause : vous l'activerez quand vous en aurez besoin pour comprendre	☐

Étape 1 - Démarrer le Fournil de Serris

Durée indicative : 15 min

À quoi ça sert · EXPLIQUER

Le Fournil de Serris vend en ligne ses pains et viennoiseries : le client commande, choisit une heure de retrait, la boulangère prépare la commande et un SMS prévient le client. L'application **marche**. Mais elle a été écrite vite, dans un seul fichier, et chaque évolution fait peur.

Route	Ce qu'elle fait
`GET /produits`	Liste le catalogue (prix en centimes, stock).
`POST /commandes`	Passe une commande : vérifie quantités, stock, créneau, calcule le total et la remise.
`GET /commandes/:id`	Consulte une commande.
`PATCH /commandes/:id/prete`	La boulangère marque la commande prête : SMS au client.

Ce que vous tapez · MODIFIER

Terminal, dans le dossier du kit

```
npm install
npm run dev
```

Ouvrez `requetes.http`, remplacez la date de retrait par **demain matin 8h30** (format `2026-10-20T08:30:00`), puis envoyez les requêtes une par une.

Ce que vous devez voir · TESTER

Ce que vous devez voir

Le terminal affiche `Fournil de Serris sur http://localhost:3000`.

`POST /commandes` répond **201** avec `"id": "C0001"`, `"total": 980` (2 baguettes + 6 croissants = 9,80 €) et `"statut": "EN_ATTENTE"`.

`PATCH /commandes/C0001/prete` répond 200 et le terminal affiche `[SMS] 0612345678 : votre commande C0001 est prête !`.

Le piège

La date de retrait passée. Avec la date d'exemple, vous obtenez 400 « Retrait possible au plus tôt 2 h après la commande ». C'est une règle métier, pas un bug.

Le fichier de données modifié. Chaque commande est écrite dans `data/db.json`. Pour repartir de zéro : remettez `"commandes": []` et les stocks d'origine.

À vous

Passez une commande de 2 tartes aux pommes. Quel total obtenez-vous, et pourquoi n'est-ce pas 3 600 ?

Étape 2 - Lire le code historique

Durée indicative : 25 min · en binôme

À quoi ça sert · EXPLIQUER

Avant de réorganiser un code, il faut savoir **ce que fait chaque ligne et pour qui**. Dans `src/main.ts`, chaque route mélange trois sortes de code :

Sorte de code	Question qu'il répond	Exemple dans `main.ts`
Présentation	Comment le monde extérieur nous parle ?	`req.body`, `res.status(400).json(…)`
Métier	Quelle est la règle de la boulangerie ?	Quantité de 1 à 20, remise de 10 % dès 30 €
Infrastructure	Où et comment on stocke, on envoie ?	`fs.readFileSync`, `console.log("[SMS] …")`

Ce que vous tapez · MODIFIER

1. Imprimez mentalement trois surligneurs : **bleu** = présentation, **vert** = métier, **orange** = infrastructure.
2. Dans `src/main.ts`, ajoutez au-dessus de chaque bloc un commentaire `// [P]`, `// [M]` ou `// [I]`.
3. Pour la route `POST /commandes`, comptez les lignes de chaque couleur.

Ce que vous devez voir · TESTER

Ce que vous devez voir

La route `POST /commandes` contient les trois couleurs **entrelacées** : une règle métier (stock suffisant) se trouve entre une lecture de fichier et une réponse HTTP.

Aucune ligne ne peut être testée sans démarrer Express **et** lire le vrai fichier `data/db.json`.

Le piège

Classer `res.status(409)` en métier. Le code 409 est un choix de présentation. La règle métier, c'est « on ne vend pas plus que le stock ».

Oublier le temps. `Date.now()` est une dépendance à l'infrastructure (l'horloge du serveur), pas une règle.

À vous

La boulangère veut passer de `data/db.json` à PostgreSQL. Listez toutes les lignes à modifier. Combien de routes sont touchées, et combien de règles métier risquez-vous de casser au passage ?

Étape 3 - Les trois couches

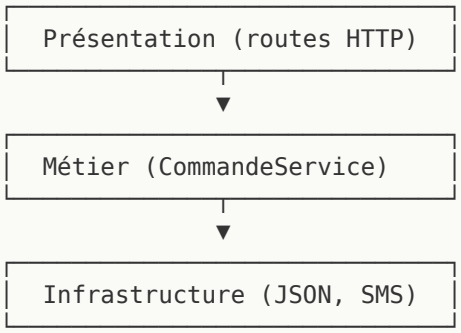
Durée indicative : 25 min

À quoi ça sert · EXPLIQUER

Une **architecture en couches** range le code par responsabilité. Chaque couche n'appelle que la couche du dessous. Analogie : au restaurant, le serveur (présentation) prend la commande, le chef (métier) décide de la recette, le fournisseur et la chambre froide (infrastructure) apportent et stockent les ingrédients. Le serveur ne cuisine pas.

Couche	Responsabilité	Ne fait jamais	Dans le Fournil
Présentation	Recevoir une requête, la traduire, renvoyer une réponse	Décider d'une règle métier	`presentation/commandeRoutes.ts`
Métier	Appliquer les règles de la boulangerie	Connaître HTTP, JSON, SQL	`metier/CommandeService.ts`
Infrastructure	Stocker, lire, envoyer (fichier, base, SMS)	Décider d'une règle métier	`infrastructure/*.ts`

Architecture en couches : chaque flèche = « importe »



Ce que vous tapez · MODIFIER

Créez l'arborescence cible (vide pour l'instant) :

Arborescence cible

```
src/
├── presentation/   commandeRoutes.ts
├── metier/         CommandeService.ts, ErreurMetier.ts
├── infrastructure/ jsonDb.ts, JsonProduitRepository.ts,
│                   JsonCommandeRepository.ts, SmsConsole.ts
├── app.ts          crée l'application Express
└── main.ts         démarre le serveur
```

Ce que vous devez voir · TESTER

Ce que vous devez voir

Trois dossiers vides dans `src/`, et `main.ts` toujours fonctionnel (vous n'avez encore rien déplacé).

Le piège

La règle cachée dans la route. Un `if (total >= 3000)` dans un fichier de `presentation/` est une règle métier égarée. Si elle y reste, l'application mobile qui appellera un jour le métier sans passer par Express ne fera pas la remise.

À vous

Classez dans la bonne couche : la validation du format JSON reçu ; « on ne retire qu'une commande prête » ; l'envoi d'un SMS via un fournisseur ; le calcul du prochain identifiant `C0002` ; le choix du code 404. Un de ces cas se discute : lequel, et pourquoi ?

Étape 4 - TP - Découper en couches

Durée indicative : 60 min · en binôme

À quoi ça sert · EXPLIQUER

On déplace le code **sans changer le comportement** : mêmes routes, mêmes réponses, mêmes erreurs. Le métier lève des `ErreurMetier` avec un code (`INVALIDE`, `INTROUVABLE`, `CONFLIT`) ; la présentation les traduit en statut HTTP. C'est la frontière entre les deux couches.

Ce que vous tapez · MODIFIER

1. L'infrastructure. L'accès au fichier et le premier repository (un *repository* = l'objet qui range et retrouve des données) :

src/infrastructure/jsonDb.ts

```
import fs from 'node:fs';

// Accès brut au fichier JSON qui sert de base de données.
const DB_FILE = process.env.DB_FILE ?? 'data/db.json';

export function lireDb(): any {
  return JSON.parse(fs.readFileSync(DB_FILE, 'utf-8'));
}

export function ecrireDb(db: any): void {
  fs.writeFileSync(DB_FILE, JSON.stringify(db, null, 2));
}
```

src/infrastructure/JsonProduitRepository.ts

```
import { lireDb, ecrireDb } from './jsonDb';

export class JsonProduitRepository {
  tous(): any[] {
    return lireDb().produits;
  }

  trouverParId(id: string): any | undefined {
    return lireDb().produits.find((p: any) => p.id === id);
  }

  enregistrer(produit: any): void {
    const db = lireDb();
    db.produits = db.produits.map((p: any) => (p.id === produit.id ? produit : p));
    ecrireDb(db);
  }
}
```

2. Le métier. L'erreur métier et le début du service :

src/metier/ErreurMetier.ts

```
export type CodeErreur = 'INVALIDE' | 'INTROUVABLE' | 'CONFLIT';

export class ErreurMetier extends Error {
  constructor(message: string, readonly code: CodeErreur) {
    super(message);
  }
}
```

src/metier/CommandeService.ts - début

```
// Couche métier : les règles de la boulangerie.
// Problème à voir en séance 1 : ce fichier IMPORTE l'infrastructure.
import { JsonProduitRepository } from '../infrastructure/JsonProduitRepository';
import { JsonCommandeRepository } from '../infrastructure/JsonCommandeRepository';
import { SmsConsole } from '../infrastructure/SmsConsole';
import { ErreurMetier } from './ErreurMetier';

export class CommandeService {
  private produits = new JsonProduitRepository();
  private commandes = new JsonCommandeRepository();
  private sms = new SmsConsole();

  listerProduits() {
    return this.produits.tous();
  }

  passerCommande(demande: { client: string; telephone: string; lignes: any[];
retrait: string }) {
    const { client, telephone, lignes, retrait } = demande;
    if (!client || !telephone) throw new ErreurMetier('Nom et téléphone
obligatoires', 'INVALIDE');
    if (!lignes || lignes.length === 0) throw new ErreurMetier('Commande vide',
'INVALIDE');
    // … déplacez ici le reste des règles de POST /commandes (stock, créneau, remise)
  }
}
```

3. La présentation. La route ne fait plus que traduire :

src/presentation/commandeRoutes.ts - début

```
// Couche présentation : traduit HTTP <-> appels au métier. Aucune règle ici.
import { Router, Response } from 'express';
import { CommandeService } from '../metier/CommandeService';
import { ErreurMetier } from '../metier/ErreurMetier';

const STATUT_HTTP = { INVALIDE: 400, INTROUVABLE: 404, CONFLIT: 409 } as const;

function repondreErreur(res: Response, e: unknown) {
  if (e instanceof ErreurMetier) return
res.status(STATUT_HTTP[e.code]).json({ erreur: e.message });
  throw e;
}

export function commandeRoutes(): Router {
  const router = Router();
  const service = new CommandeService();

  router.get('/produits', (req, res) => {
    res.json(service.listerProduits());
  });

  router.post('/commandes', (req, res) => {
    try {
      res.status(201).json(service.passerCommande(req.body));
    } catch (e) {
      repondreErreur(res, e);
    }
  });

  // … GET /commandes/:id et PATCH /commandes/:id/prete : à vous
  return router;
}
```

4. Le démarrage.

src/app.ts

```
import express from 'express';
import { commandeRoutes } from './presentation/commandeRoutes';

export const app = express();
app.use(express.json());
app.use(commandeRoutes());
```

src/main.ts (remplace tout l'ancien contenu)

```
import { app } from './app';

app.listen(3000, () => console.log('Fournil de Serris sur http://localhost:3000'));
```

La suite est à vous : `JsonCommandeRepository` (sur le modèle de `JsonProduitRepository`, avec `prochainId()`), `SmsConsole` (une méthode `envoyer(telephone, message)`), puis `consulterCommande` et `marquerPrete` dans le service et leurs routes.

Ce que vous devez voir · TESTER

Ce que vous devez voir

Les 4 requêtes de `requetes.http` donnent **exactement** les mêmes réponses qu'à l'étape 1 (statuts, corps, SMS dans le terminal).

`npm run typecheck` ne signale aucune erreur.

Plus aucun `fs` ni `console.log` dans `presentation/` et `metier/`.

Le piège

L'erreur avalée. Si `repondreErreur` ne reconnaît pas l'erreur, Express renvoie 500 au lieu de 400 : vérifiez que le service lève bien `new ErreurMetier(…)`, pas `new Error(…)`.

Le stock relu trop tôt. Décrémentez le stock **après** toutes les vérifications, sinon une commande refusée aura quand même vidé les rayons.

À vous

Faites un commit : `git init` (si besoin), `git add .`, `git commit -m "Découpage en couches"`. Puis ajoutez une route `GET /commandes/:id/total-euros` qui renvoie `{ "total": "10,00 €" }`. Dans quelle couche mettez-vous la mise en forme en euros, et pourquoi ?

Étape 5 - Tracer les dépendances

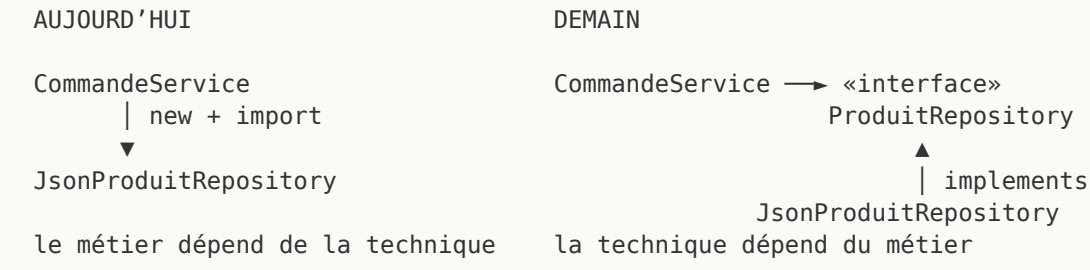
Durée indicative : 25 min

À quoi ça sert · EXPLIQUER

Regardez les `import` de `CommandeService.ts` : le métier **importe** l'infrastructure et crée lui-même ses repositories avec `new`. Conséquence : impossible d'utiliser le service sans le vrai fichier JSON, impossible de le tester sans écrire sur le disque, impossible de remplacer le JSON par PostgreSQL sans modifier le métier.

Le **principe d'inversion des dépendances** (DIP, le D de SOLID) dit : *les modules de haut niveau (le métier) ne doivent pas dépendre des modules de bas niveau (la technique) ; les deux dépendent d'abstractions*. Et c'est le haut niveau qui **possède** l'abstraction.

Aujourd'hui → demain (séance 2)



Ce que vous tapez · MODIFIER

1. Dessinez sur papier le graphe des `import` de `src/` : une boîte par fichier, une flèche par import.
2. Entourez en rouge chaque flèche qui va du métier vers l'infrastructure.
3. Essayez d'écrire un test du calcul de remise : créez `tests/remise.test.ts` qui appelle `new CommandeService().passerCommande(…)`.

Ce que vous devez voir · TESTER

Ce que vous devez voir

Trois flèches rouges : `CommandeService` → `JsonProduitRepository`, `JsonCommandeRepository`, `SmsConsole`.

Votre test écrit une vraie commande dans `data/db.json`, et il **échoue ou réussit selon l'heure** : le créneau « dans 2 h » dépend de `Date.now()`.

Le piège

Le test qui passe le matin. Un test qui commande pour 11h passe à 8h et échoue à 10h. Un test qui dépend de l'horloge, du disque ou du réseau n'est pas un test unitaire : c'est un signe que le métier dépend de l'infrastructure.

À vous

Supprimez votre test et `git checkout -- data/db.json`. Écrivez en une phrase ce qu'il faudrait pour que `CommandeService` utilise un faux repository pendant les tests.

Étape 6 - SOLID à l'échelle d'une architecture

Durée indicative : 25 min

À quoi ça sert · EXPLIQUER

Vous connaissez SOLID pour une classe. Les mêmes principes s'appliquent aux **modules et aux couches** : c'est ce qui sépare une application qu'on fait évoluer pendant dix ans d'une application qu'on réécrit tous les deux ans.

Principe	Pour une classe	Pour une architecture	Dans le Fournil
S - Responsabilité unique	Une classe, une raison de changer	Un module, une raison de changer	Changer de fournisseur SMS ne touche que `infrastructure/`
O - Ouvert / fermé	Étendre sans modifier	Ajouter un adapter sans modifier le métier	PostgreSQL = un nouveau repository, zéro ligne du métier modifiée
L - Substitution	Une sous-classe remplace sa mère	Toute implémentation respecte le contrat	Le repository en mémoire se comporte comme le JSON
I - Ségrégation des interfaces	Pas de méthode inutile imposée	Des ports petits, taillés pour un besoin	Pas d'interface `BaseDeDonnees` à 15 méthodes
D - Inversion des dépendances	Dépendre d'abstractions	Le métier définit les interfaces, la technique les implémente	Les flèches rouges de l'étape 5 doivent s'inverser

Ce que vous tapez · MODIFIER

Pour chaque évolution demandée par la boulangère, notez **les fichiers touchés** dans votre version en couches, puis **le principe SOLID** qui limiterait ce nombre :

1. Envoyer les SMS par un vrai fournisseur au lieu de la console.
2. Ouvrir le Click & Collect par une application mobile (sans Express).
3. Tester la règle « retrait entre 7h et 19h30 » en moins d'une seconde.

Ce que vous devez voir · TESTER

Ce que vous devez voir

Le cas 1 est déjà bien isolé (un seul fichier, mais le métier doit changer son `new`). Les cas 2 et 3 touchent le métier ou sont impossibles : la couche métier n'est pas encore indépendante.

Le piège

L'interface pour l'interface. Créer une interface par classe « parce que SOLID » ne sert à rien. Une abstraction se justifie quand elle protège le métier d'un détail technique qui va changer (stockage, envoi, horloge, API externe).

À vous

Donnez un exemple d'abstraction **inutile** dans le Fournil, et un exemple d'abstraction **indispensable**. Justifiez en une phrase chacune.

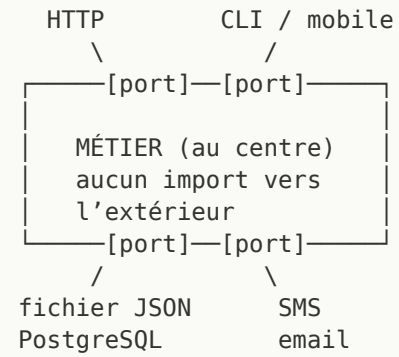
Étape 7 - Les limites de l'architecture en couches

Durée indicative : 15 min

À quoi ça sert · EXPLIQUER

L'architecture en couches range bien le code, mais **la base de données reste tout en bas, donc au fondement** : tout dépend d'elle. L'**architecture hexagonale** (Alistair Cockburn, 2005, aussi appelée *ports and adapters*) retourne le schéma : le métier au centre, la technique autour, branchée par des prises.

L'idée de l'hexagone (séance 2)



Ce que vous tapez · MODIFIER

Aucun code : relisez votre `CommandeService` et listez ce dont il a **besoin** (pas ce qu'il utilise) : « retrouver un produit », « enregistrer une commande », « connaître l'heure », « prévenir le client ».

Ce que vous devez voir · TESTER

Ce que vous devez voir

Une liste de 4 à 6 besoins écrits en **langage métier**, sans aucun mot technique (pas de « JSON », « fichier », « console »). Ce seront vos **ports** à la séance 2.

Le piège

Le besoin écrit en technique. « lireFichierJson » n'est pas un besoin du métier ; « trouver un produit par son identifiant » en est un.

À vous

Pourquoi dit-on qu'une architecture en couches classique est « centrée sur la base de données » ? Répondez en une phrase en vous appuyant sur le sens des flèches de l'étape 5.

Mémo express

Les notions de la séance

Notion	Sert à	Exemple
Couche présentation	Traduire le monde extérieur en appels au métier	`res.status(201).json(commande)`
Couche métier	Porter les règles, indépendamment de la technique	`if (total >= 3000)` … dans le service
Couche infrastructure	Stocker, lire, envoyer	`fs.writeFileSync`, envoi SMS
Repository	Ranger et retrouver des objets métier	`trouverParId(id)`, `enregistrer(c)`
ErreurMetier	Signaler une règle violée sans parler HTTP	`new ErreurMetier("Commande vide", "INVALIDE")`
DIP	Le métier possède les abstractions, la technique les implémente	`JsonProduitRepository implements ProduitRepository`

Les commandes

Action	Commande
Installer les dépendances	`npm install`
Lancer l'API (redémarre à chaque sauvegarde)	`npm run dev`
Vérifier les types	`npm run typecheck`
Annuler les modifications de la base	`git checkout -- data/db.json`
Palette de commandes VS Code	`Ctrl+Maj+P` / `Cmd+Maj+P`

Les 5 réflexes à garder

- Refactorer = déplacer sans changer le comportement : comparez les réponses avant et après.
- Une règle métier ne connaît ni HTTP, ni fichier, ni horloge système.
- Lisez les `import` : ils disent qui dépend de qui.
- Une abstraction se justifie par un détail technique qui va changer.
- Commitez à chaque étape qui fonctionne.